Lecture 4 - Java Multi-Threading

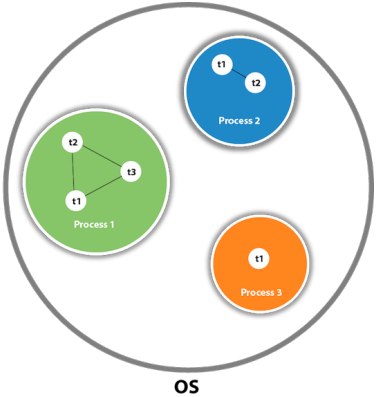
Guiliang Liu

The Chinese University of Hong Kong, Shenzhen

CSC-1004: Computational Laboratory Using Java
Course Page: [Click]

Multitasking in Java

Multitasking is a process of executing multiple tasks simultaneously.



Multitasking can be achieved in two ways:

Multitasking in Java

Process-based Multitasking (Multiprocessing)

- Each process has an address in memory. It allocates a separate memory area.
- A process is heavy-weight.
- Cost of communication between the process is high. Switching from one process to another requires some time for saving and loading registers, memory maps, updating lists, etc.

Multitasking in Java

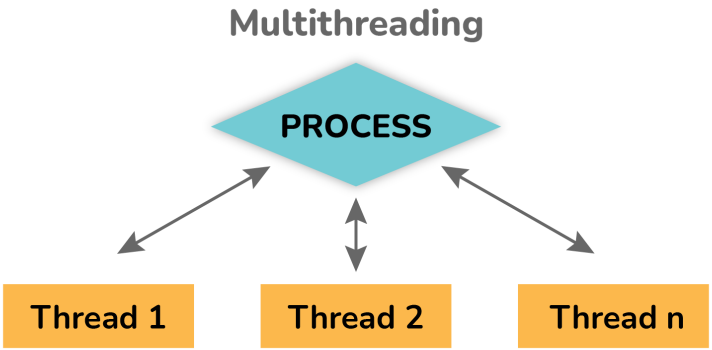
Thread-based Multitasking (Multithreading)

What is Thread in java?

- A thread is a lightweight subprocess, the smallest unit of processing.
- Threads are independent. If there is an exception occurs in one thread, it doesn't affect other threads. It uses a shared memory area.
- Threads use a **shared memory** area. They don't allocate separate memory areas so save memory, and context-switching between the threads takes less time than the process.

Multithreading in Java

Multithreading in Java is a process of executing multiple threads simultaneously.



InterviewBit

圳)
g Kong, Shenzhen

Multithreading in Java

Advantages of Java Multithreading.

- It doesn't block the user because threads are independent and you can perform multiple operations at the same time.
- You can perform many operations together, so it saves time.
- Threads are independent, so it doesn't affect other threads if an exception occurs in a single thread.

Life cycle of a Thread (Thread States)

In Java, a thread always exists in any one of the following states. These states are:

1. **New:** Whenever a new thread is created, it is always in the new state. For a thread in the new state, the code has not been run yet and thus has not begun its execution.

Life cycle of a Thread (Thread States)

In Java, a thread always exists in any one of the following states. These states are:

1. **New:**
2. **Active:** When a thread invokes the start() method, it moves from the new state to the active state. The active state contains two states within it: one is runnable, and the other is running.
   - 2.1 *Runnable*: A thread, that is ready to run is then moved to the runnable state. In the runnable state, the thread may be running or may be ready to run at any given instant of time. The thread scheduler provides the thread time to run.
   - 2.2 *Running*: When the thread gets the CPU, it moves from the runnable to the running state. Generally, the most common change in the state of a thread is from runnable to running and again back to runnable.

Life cycle of a Thread (Thread States)

In Java, a thread always exists in any one of the following states. These states are:

1. **New:**
2. **Active:**
3. **Blocked / Waiting:** Whenever a thread is inactive for a span of time (not permanently) then, either the thread is in the blocked state or is in the waiting state. For example, a thread (A) may want to print some data from the printer. However, at the same time, the other thread (B) is using the printer to print some data. Therefore, thread A has to wait for thread B to use the printer. Thus, thread A is in the blocked state.

Life cycle of a Thread (Thread States)

In Java, a thread always exists in any one of the following states. These states are:

1. **New:**
2. **Active:**
3. **Blocked / Waiting:**
4. **Timed Waiting:** Sometimes, waiting leads to starvation. For example, a thread (A) has entered the critical section of a code and is not willing to leave that critical section. In such a scenario, another thread (B) has to wait forever, which leads to starvation. To avoid such scenario, a timed waiting state is given to thread B. A real example of timed waiting is when we invoke the sleep(#Time) method on a specific thread.

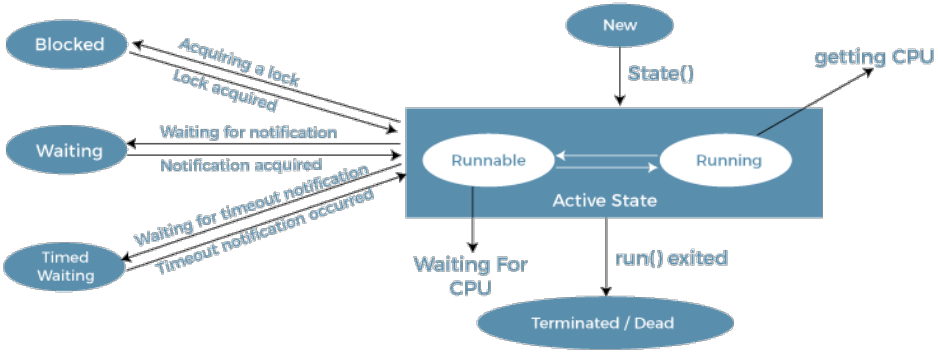
Life cycle of a Thread (Thread States)

In Java, a thread always exists in any one of the following states. These states are:

1. **New:**
2. **Active:**
3. **Blocked / Waiting:**
4. **Timed Waiting:**
5. **Terminated:** A thread reaches the termination state because of the following reasons:1) When a thread has finished its job, then it exists or terminates normally. 2) Abnormal termination: It occurs when some unusual events such as an unhandled exception or segmentation fault.

Life cycle of a Thread (Thread States)

The following diagram shows the different states involved in the life cycle of a thread.



Life Cycle of a Thread

Create Java Threads

Method 1: Create a thread in Java by implementing a Runnable interface. For example:

```
public class Main implements Runnable {
   public static void main(String[] args) {
      Main obj = new Main();
      Thread thread = new Thread(obj);
      thread.start();
      System.out.println("This code is outside of the thread");
  }
   public void run() {
      System.out.println("This code is running in a thread");
  }
}
```

Create Java Threads

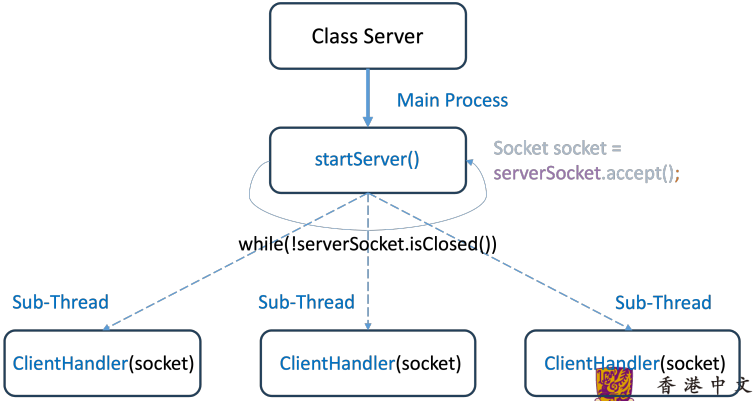
Method 2: Create a thread in Java by using the Thread Class: Thread(Runnable r, String name). For example:

```
public class Main extends Thread {
   public static void main(String[] args) {
      Main thread = new Main();
      thread.start();
      System.out.println("This code is outside of the thread");
   }
   public void run() {
      System.out.println("This code is running in a thread");
   }
}
```

Which method is better and why?

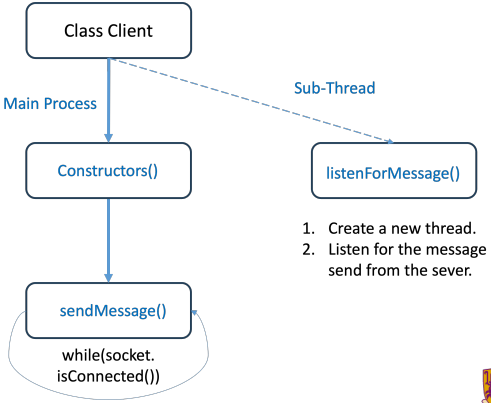
Multi-threading in Chatroom

Structure of Class Server (Please check our example code)



Multi-threading in Chatroom

Structure of Class Client (Please check our example code)



香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Question and Answering (Q&A)